

第 6, 7 回 データ構造

執筆: 安永憲司

1 動的集合と動的集合への操作

アルゴリズムが整数などの集合に対して計算を行うとき、対象とする集合が時間とともに変化することがある。例えば、要素を追加したり、削除したり、最小の要素を取り出したりする操作を繰り返す場合がある。このような集合を**動的集合** (*dynamic set*) と呼ぶ。動的集合に対する操作を効率よく実現するためのデータの表現方法、すなわち**データ構造** (*data structure*) を考えよう。

動的集合 X は、整数、実数、文字列、グラフなどのオブジェクトを要素として含むものとする。各要素には、比較や順序付けの基準となる値が対応していると考え、これを**鍵** (*key*) と呼ぶ。例えば、 X が整数の集合であれば、各要素の値そのものを鍵とみなせばよい。また、 X がグラフの集合であれば、各グラフに付けた ID、頂点数、辺数などを鍵として用いることができる。

一般には、要素そのものを直接扱うのではなく、各要素を指すポインタ x_i を用いて扱うことが多い。このとき、各ポインタ x_i があるオブジェクトを指し、そのオブジェクトに鍵 k_i が対応していると考ええる。

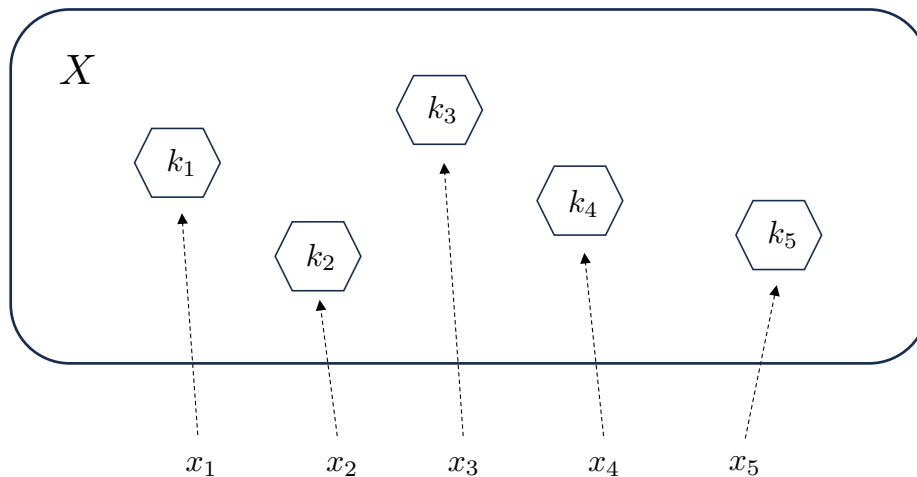


図 1 動的集合 X とその要素 $x_1, \dots, x_5$ と鍵 $k_1, \dots, k_5$

動的集合 X に対しては、様々な操作を考えることができる。実際には、すべての操作を効率よく実現する必要はなく、目的に応じて必要な操作だけを効率よく実現できればよい。代表的な操作の組として、辞書と優先度付きキューを取り上げる。

■**辞書 (dictionary)** 辞書とは、動的集合に対して以下の Member, Insert, Delete を実行できるようにしたものである。

1. Member(x, X): 要素 x が集合 X に含まれているか否かを判定する。
2. Insert(x, X): 要素 $x \notin X$ を X に加える。
3. Delete(x, X): 要素 $x \in X$ を X から取り除く。

■**優先度付きキュー** (priority queue) 優先度付きキューとは、各要素に鍵が対応しており、鍵の小さい要素ほど優先度が高いとみなす動的集合である。基本的な操作として、以下の FindMin, Insert, DeleteMin, DecreaseKey などがある。

1. FindMin(X): X 内の鍵最小の要素を返す。
2. Insert(x, X): 要素 x を X に加える。
3. DeleteMin(X): X 内の鍵最小の要素を取り除く。
4. DecreaseKey(x, k, X): 要素 $x \in X$ の鍵を k に更新する。ただし、 k は現在の鍵以下の値とする。

2 配列とヒープ

配列 (array) は最も基本的なデータ構造であり、連続したデータを格納する方法として、多くのプログラミング言語で用意されている。配列の場合、任意の場所 $A[i]$ の値を読み込むための時間は定数と考える。

```
1  #include <stdio.h>
2  int main(void) {
3      int A[3];
4      int B[3] = { 2, 4, 6 };
5      int C[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };
6      printf("%d\n", B[0]);
7      printf("%d\n", B[2]);
8      printf("%d\n", C[0][1]);
9      printf("%d\n", C[1][2]);
10     return 0;
11 }
```

優先度付きキューを効率よく実現する代表的なデータ構造として、**ヒープ** (heap) がある。ヒープは、完全二分木に要素を配置し、親子間に一定の大小関係を満たすようにしたものである。格納されている要素数 n に対し、ヒープは FindMin を計算量 $O(1)$, Insert, DeleteMin, DecreaseKey を計算量 $O(\log_2 n)$ で実現する。さらに、ヒープは配列を用いて実現できる。

2.1 グラフや根付き木に関する用語

グラフ (graph) $G = (V, E)$ は、有限個の頂点 (vertex) の集合 V と辺 (edge) の集合 E から構成される。辺 $e \in E$ が順序対 (u, v) ($u, v \in V$) で与えられるとき、 G は**有向グラフ** (directed graph), 順序なし対 $\{u, v\}$ で与えられるとき、**無向グラフ** (undirected graph) と呼ぶ。(慣例として、無向グラフにおいても (u, v) で辺を表し、 (u, v) と (v, u) は同一辺を表すものとする。)

頂点の列 $(v_0, v_1, \dots, v_k)$ が全て辺を辿る (すべての i について $(v_i, v_{i+1}) \in E$) とき、長さ k の**パス** (path) と呼び、 $v_0 = v_k$ かつ一つ以上の辺を含むとき、そのパスを**閉路** (cycle) と呼ぶ。頂点 u から v へのパスが存在するとき、 u から v に**到達可能** (reachable) といい、グラフの頂点同士がすべて到達可能なとき、**連結している** (connected) という。グラフ G の連結な部分グラフで極大なもの (他の連結部分グラフに真に含まれないもの) を G の**連結成分** (connected component) と呼ぶ。

有向グラフでは、各有向辺を無向辺に置き換えたとき連結グラフになれば**弱連結** (weakly connected) と呼ぶ。無向辺に置き換えずに、任意の頂点同士が到達可能ならば**強連結** (strongly connected) と呼ぶ。

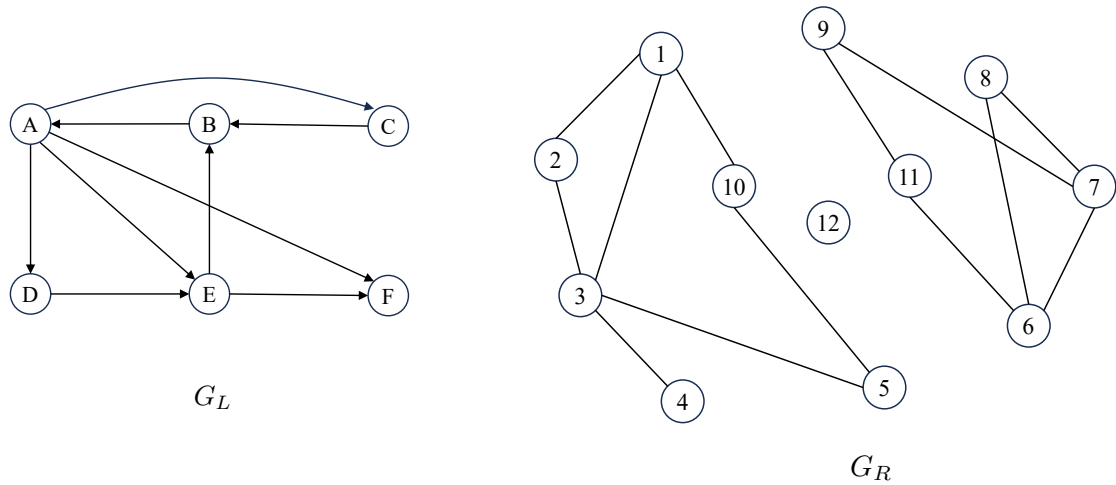


図2 グラフの例

例 2.1. 図 2 の左側のグラフ G_L は有向グラフであり, $G_L = (V_L = \{A, B, C, D, E, F\}, E_L = \{(A, C), (A, D), (A, E), (A, F), (B, A), (C, B), (D, E), (E, B), (E, F)\})$ で与えられる. グラフ G_L において, (A, D, E, F) は長さ 3 のパスであり, (A, E, B, A) は閉路である. グラフ G_L は弱連結であるが, 強連結ではない. 右のグラフ G_R は無向グラフであり, $(1, 3, 5, 10)$ は長さ 3 のパスであり, $(6, 7, 8, 6)$ は閉路である. また, G_R は連結ではなく, $\{1, 2, 3, 4, 5, 10\}$, $\{6, 7, 8, 9, 11\}$, $\{12\}$ は連結成分である.

閉路のない (acyclic) 連結無向グラフを**木 (tree)**という. 木の中で, 根 (root) と呼ばれる特別な頂点をもつものを**根付き木 (rooted tree)**という. 根付き木における頂点はノード (node) と呼ぶことが多い. 根付き木 T では, 根 r から各ノードへのパスが一意に定まる. そのため, 根を基準としてノード間に上下関係を定義できる. 根 r からノード x に至るパス上のノードを x の**祖先 (ancestor)**といい, y が x の祖先であるとき, x は y の**子孫 (descendant)**という. 根からのパスの最後の辺が (y, x) であるとき, y は x の**親 (parent)**, x は y の**子 (child)**という. 同じ親をもつノード同士は**兄弟 (sibling)**といい, 子をもたないノードは**葉 (leaf)**と呼ばれる. 根 r からノード x への単純パス (同じ頂点を 2 度以上通らないパスのこと) の長さを x の**深さ (depth)**という. ノード x から葉への単純パスの長さの最大値を x の**高さ (height)**といい, 木の高さとはその根の高さである.

二分木 (binary tree)とは, 各ノードが高々 2 つの子をもつ根付き木のことであり, その子は左の子 (left child)・右の子 (right child) と呼ばれ区別される. 二分木において, 根からの深さが最大でない部分のノードがすべて埋まっており, 深さ最大のノードが左から順に埋まっているとき, **完全二分木 (complete binary tree)**と呼ぶ. ヒープでは, この意味での完全二分木を用いる.

例 2.2. 図 3 は左右とも二分木である. 頂点 1, A をそれぞれ根と考えると根付き木である. 左の木において, 3 は 1 の子であり, 4 と 5 の親である. 左の木の葉は 2, 5, 6 であり, 木の高さは 3 である. 右のグラフは, 深さ最大のノードが左から順に埋まっているので, 完全二分木だと考える.

2.2 ヒープ条件とヒープの操作

ヒープは完全二分木であり, 根以外のすべてのノード v とその親 $p(v)$ に対し,

$$\text{ヒープ条件: } p(v).key \leq v.key$$

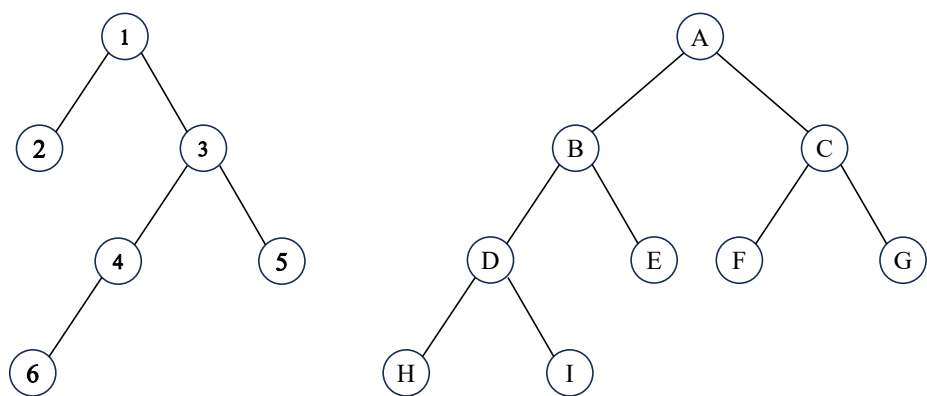


図3 二分木の例

を満たすように各ノード v に X の要素が割り当てられたものである。ここで、 $v.key$ はノード v に対応する要素の鍵を表す。（ヒープ条件の大小関係を逆にしたものをも考えてもよい。上記は、鍵の値が小ささが優先度の高さに対応している。）完全二分木の各ノードを幅優先順（breadth-first order）に番号づけすれば、配列で表現できる。サイズ n の配列 A を用意し、 $A[i]$ には幅優先順で i 番目のノードに割り当てられている要素の鍵を格納する。（図4を参照。根は1番目であり、 $A[1]$ は根の要素の鍵である。）

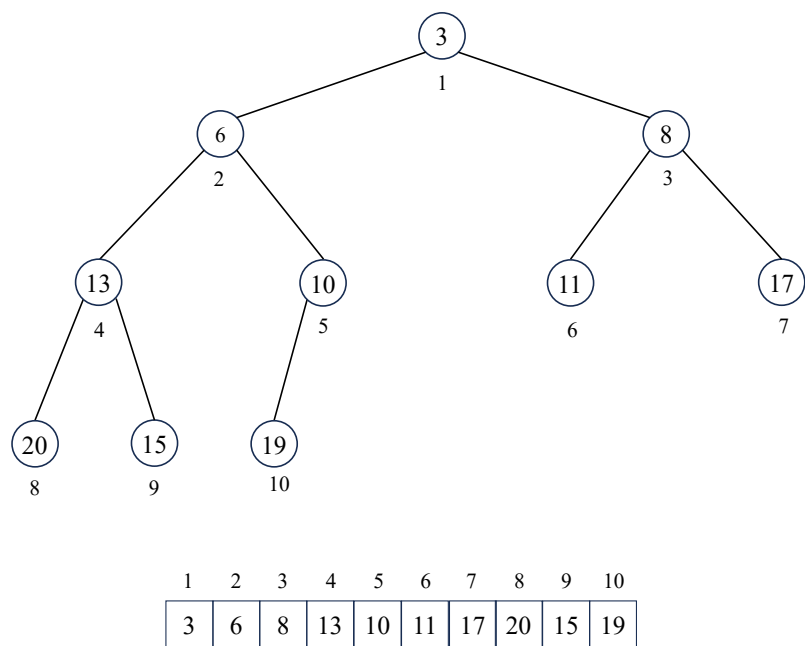


図4 ヒープとその配列表現（図3とは異なり、木のノードの中の値は鍵であり、ノードの外の値が頂点番号である。）

ヒープに含まれる要素数が n のとき、高さは $d = O(\log_2 n)$ である。以下では、FindMin, Insert, DeleteMin, DecreaseKey の実現方法とその計算量を見る。

■FindMin(X) ヒープ条件より、根にある要素 ($A[1]$ の要素) が最小の鍵をもつため、FindMin は計算量 $O(1)$ で実行できる。

■Insert(x, X) 完全二分木の幅優先順に $n + 1$ 番目のノード v を追加し、要素 x の鍵を $v.key$ に対応させる。その後、ヒープ条件を満たすように以下の「篩い上げ」を行えばよい（図5参照）。

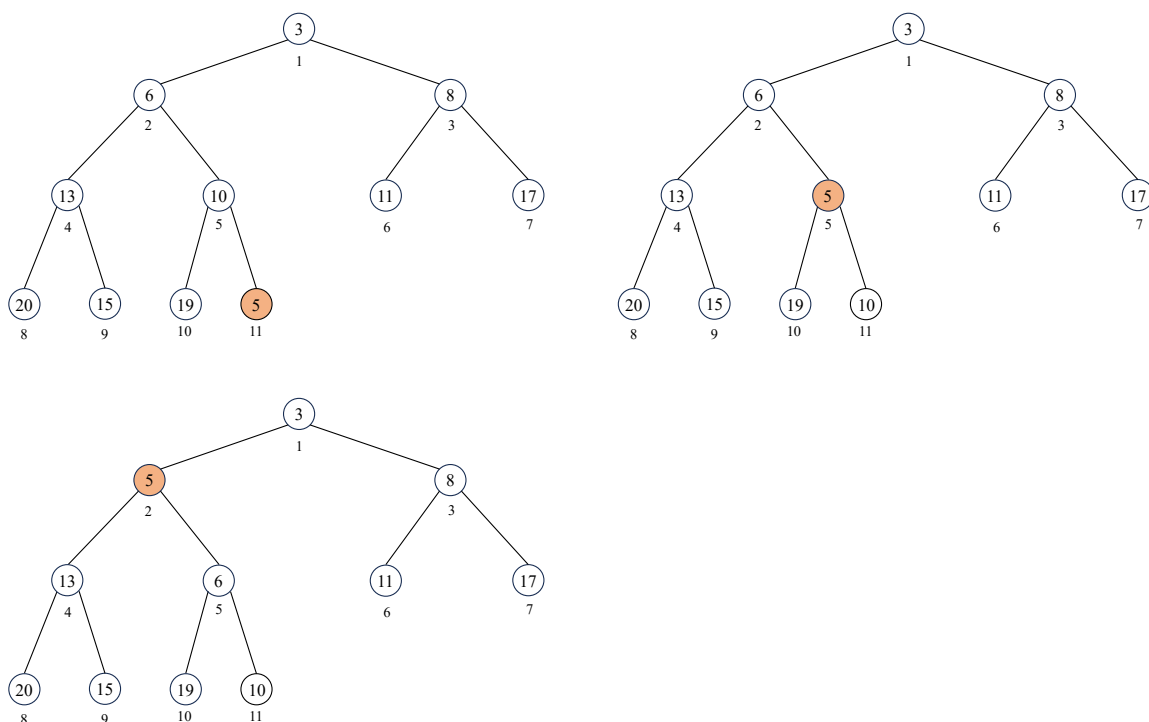


図5 ヒープの篩い上げ

v からの篩い上げ

1. v が根である, または $p(v).key \leq v.key$ ならば停止.
2. そうでないとき, $p(v)$ と v の要素を入れ替え, $v = p(v)$ として 1 に戻る.

ノード v とその親 $p(v)$ の鍵の大きさを比較する必要があるが, ノード v がヒープの i 番目のノードであるとき, v に対応する要素の鍵は $A[i]$ であり, その親の鍵は $A[\lfloor i/2 \rfloor]$ であるため, 配列を使って比較できる. $\text{Insert}(x, X)$ は以下のように実行できる. 計算量は木の高さに比例し, $O(\log_2 n)$ である.

1. 幅優先順で最後のノードの次にノード v をつくり, v に要素 x を対応させる.
2. v からの篩い上げを行う.

■ $\text{DecreaseKey}(x, k, X)$ 篩い上げを用いれば以下のように実行できる. 要素 x に対応するノード v が既知の場合, 計算量は $O(\log_2 n)$ である. (ノード v がわからないとき, 計算量 $O(\log_2 n)$ で実現できるとは言えない.)

1. 要素 x に対応するノード v に対し, 鍵 $v.key$ を k にする.
2. v からの篩い上げを行う.

■ $\text{DeleteMin}(X)$ 根の要素は鍵の値が最小なので削除し, そこに最後のノードの要素を移し, 最後のノードは取り除く. その後, 根 r に対し, ヒープ条件を満たすように以下の「篩い下げ」を行う (図 6 参照).

v からの篩い下げ (v を根とする部分木のヒープ化)

1. v が葉である, または v のすべての子 w について $v.key \leq w.key$ を満たすならば停止.
2. そうでないとき, v の子のうち鍵の値が最小のものを w とし, w と v の要素を入れ替え, $v = w$ として 1 に戻る.

ノード v がヒープの i 番目のノードであるとき, v に対応する要素の鍵は $A[i]$, v の左の子の鍵は $A[2i]$, 右の子の鍵は $A[2i+1]$ であり, 配列を使って比較できる. $\text{DeleteMin}(X)$ は以下のように実行できる. 計

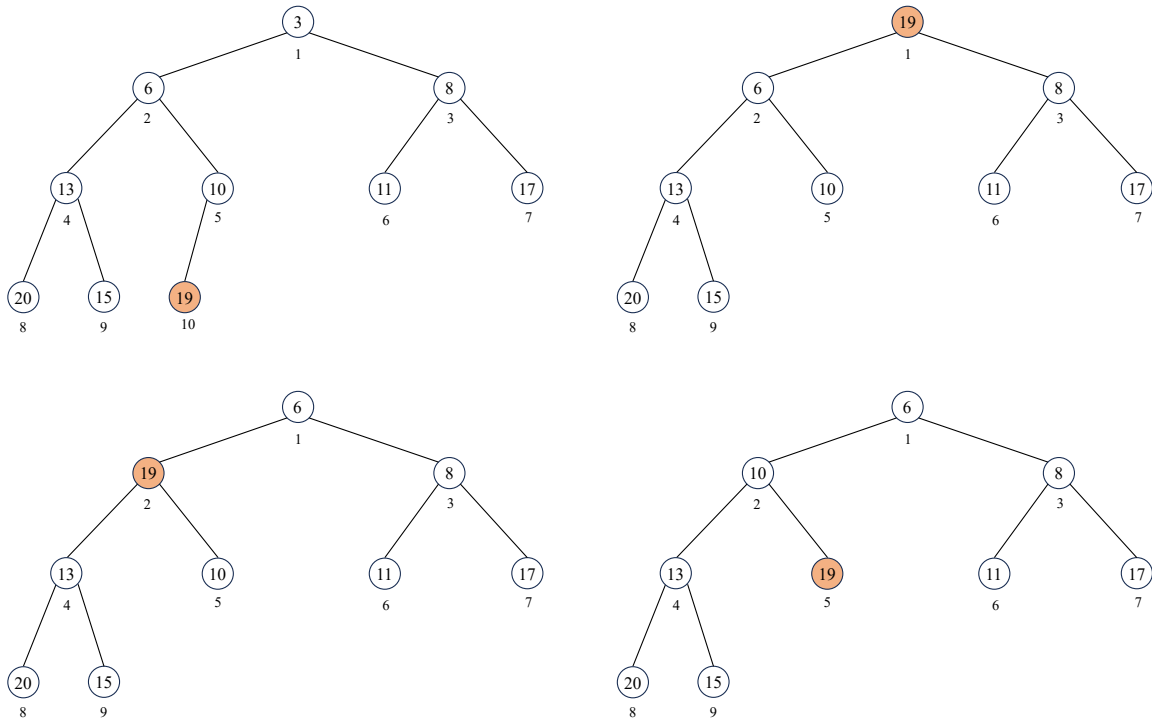


図6 DeleteMin におけるヒープの篩い下げ

算量は $O(\log_2 n)$ である。

1. 幅優先順で最後のノードの要素を根に移し、最後のノードは取り除く。
2. 根 r からの篩い下げを行う。

2.3 ヒープ構成の計算量

集合 X が与えられたとき、それに対応するヒープの構成は、空のヒープから始め、各 $x \in X$ に対し Insert を実行すればよいので、計算量 $O(n \log_2 n)$ でできる。ここでは、計算量 $O(n)$ で実現する方法を紹介する。各要素を完全二分木の各ノードに割り振り、その後、幅優先順の逆順に、子をもつノードに対して篩い下げを行う。

1. 集合 X の各要素を任意の順番に完全二分木の各ノードに割り振る ($|X| = n$ とする)。
2. 幅優先順で $\lfloor n/2 \rfloor$ 番目にあるノード v から逆順に、 v からの篩い下げを行う。

半分以降のノードは子をもたないので、 $\lfloor n/2 \rfloor$ から始めている。深さの大きな順に篩い下げを行うため、ヒープを構成できる。あとは計算量を解析しよう。

木の高さを $d = \lfloor \log_2 n \rfloor$ とする。ノード v の篩い下げの計算量は、その深さを $\text{depth}(v)$ で表すとき、 $O(d - \text{depth}(v))$ である。このとき、合計計算量は、幅優先順で i 番目のノードを v_i と表せば、

$$\begin{aligned}
 \sum_{i=1}^{\lfloor n/2 \rfloor} (d - \text{depth}(v_i)) &\leq \sum_{j=0}^d (d - j) \cdot |\{v : \text{depth}(v) = j\}| \\
 &\leq \sum_{j=0}^d (d - j) 2^j \\
 &= d(2^{d+1} - 1) - 2(d2^d - 2^d + 1) = 2^{d+1} - d - 2 \\
 &\leq 2n
 \end{aligned}$$

と抑えられる。ここで、 $\sum_{j=0}^d j2^j = 2(d2^d - 2^d + 1)$ を使っている。したがって、計算量は $O(n)$ である。

2.4 ヒープソート

ヒープを使えば、最悪の場合の計算量が $O(n \log_2 n)$ のソートアルゴリズムが得られる。手順は以下の通りである。

1. n 個の要素から 2.3 節の方法によりヒープを構成する（各要素の値をそのまま鍵とする）。
2. FindMin と DeleteMin を $n - 1$ 回実行する。

ヒープを配列で実現する場合、手順 2 の i 回目の実行で配列の $n - i + 1$ 番目に空きが出るので、FindMin で得た要素を空いた部分に格納すればよい。ここでは最小ヒープを使っているため、取り出した要素を配列の後ろから順に格納すると、配列の先頭から大きい順に要素が並ぶ。最大ヒープを用いれば、同様にして小さい順に並べることもできる。

このように、ヒープソートは配列以外の補助的な記憶領域をほとんど使わずに実行できる。このようなアルゴリズムは、**その場アルゴリズム** (*in-place algorithm*) と呼ばれる。アルゴリズムの計算量としては、主に**時間計算量** (*time complexity*) を気にすることが多いが、**空間計算量・領域計算量** (*space complexity*) も重要である。ヒープソートは、追加の記憶領域が定数個の変数程度で済むという意味で、空間効率のよいソートアルゴリズムである。

2.5 ヒープクイズ

クイズ 2.1. 最小ヒープについて正しいものをすべて選べ。

1. 根にある要素は、ヒープ全体で鍵が最小の要素である。
2. 鍵が最大となる要素の少なくとも一つは、必ず葉にある。
3. 親の鍵は、その子の鍵以下である。
4. 配列で表したとき、 $A[1] \leq A[2] \leq \dots \leq A[n]$ が必ず成り立つ。

クイズ 2.2. 要素数 n の最小ヒープの各操作の計算量として正しいものはどれか。

	操作	計算量		操作	計算量		操作	計算量
1.	FindMin	$O(\log_2 n)$	2.	FindMin	$O(1)$	3.	FindMin	$O(1)$
	Insert	$O(\log_2 n)$		Insert	$O(\log_2 n)$		Insert	$O(\log_2 n)$
	DeleteMin	$O(\log_2 n)$		DeleteMin	$O(\log_2 n)$		DeleteMin	$O(1)$
	DecreaseKey	$O(\log_2 n)$		DecreaseKey	$O(\log_2 n)$		DecreaseKey	$O(\log_2 n)$
	ヒープ構成	$O(n \log_2 n)$		ヒープ構成	$O(n)$		ヒープ構成	$O(n)$

クイズ 2.3. 要素数 n の最小ヒープにおいて、FindMin と DeleteMin を $O(1)$ 回使って実現できる操作を以下よりすべて選べ。ただし、それ以外の計算量は $O(1)$ とする。

1. 10 番目に小さい鍵をもつ要素を見つける。
2. 鍵が最大の要素を見つける。
3. 中央値の鍵をもつ要素を見つける。
4. 上記のいずれもできない。

3 二分探索

集合 X に対し、ある要素 x が X に含まれているか否かを判定するアルゴリズムを考える。これを高速に実現するのが**二分探索** (*binary search*) であり、データ構造としては配列で実現できる。簡単のため、 X は順序集合とする。

集合 X の各要素が配列 A に、

$$A[1] \leq A[2] \leq \cdots \leq A[n]$$

を満たすようにソートされて格納されていることが前提である。以下がそのアルゴリズムである。

二分探索

1. $i = 1, j = n$ とする
2. $i > j$ であれば $x \notin X$ と答えて停止
3. $k = \lfloor (i + j) / 2 \rfloor$ とする
 - (a) $A[k] = x$ なら $x \in X$ と答えて停止
 - (b) $A[k] < x$ なら $i = k + 1$ として 2 に戻る
 - (c) $A[k] > x$ なら $j = k - 1$ として 2 に戻る

計算量を考える。手順 3 で $A[k] < x$ のとき、 $A[1]$ から $A[k]$ までに x はないことが、 $A[k] > x$ のときは、 $A[k + 1]$ から $A[n]$ までにないことが保証される。探索の範囲は毎回半分になるため、 $\lfloor \log_2 n \rfloor + 1$ 回の比較でアルゴリズムは終了する。したがって、計算量は $O(\log_2 n)$ である。

二分探索は非常にシンプルなアルゴリズムであり、計算量は $O(\log_2 n)$ と非常に小さく、極めて有用である。一方で、プログラミング実装においてバグが入りやすいので、実装する場合は正しく実現できているかどうか、よく確認しよう。よく知られたバグの原因は、手順 3 で、

```
k = (i + j) / 2;
```

とコーディングすることである。これ自体は間違いではないが、`int` 型が 32 ビットだと、 $2^{31} - 1$ までの整数しか扱えないため、整数のオーバーフローが起きる可能性がある。 $i = 1$ と $j = 2^{31} - 1$ からスタートして、見つけたい場所が $A[j]$ に近かったとする。このとき、1 回目の繰返しで $k = 2^{30}$, $i = 2^{30} + 1$ となり、2 回目の繰返しで、

$$k = \left\lfloor \frac{(2^{30} + 1) + (2^{31} - 1)}{2} \right\rfloor$$

を計算するが、足し算をしたところでオーバーフローが起きる。このバグをなくすための工夫は、

```
k = i + (j - i) / 2;
```

と書くことである。

他にも、ループを終了して停止するための条件を正確に記述しないと、簡単に無限ループに陥ってしまう。

以下は、間違いを含む二分探索のコードである。 $A[k] < x$ のとき、 i を k に更新している。

```

1  int binary_search_wrong( int A[], int n, int x ) {
2      int i = 0;
3      int j = n - 1;
4      while ( i <= j ) {
5          int k = ( i + j ) / 2;
6          if ( A[k] == x ) return k;
7          else if ( A[k] < x ) i = k;
8          else j = k - 1;
9      }
10     return -1;
11 }

```

ループ不変式を利用して、このプログラムの間違いを見つけてみよう。今回は条件式（不変式）を、

配列 A に x が存在するなら、 $A[i...j]$ の中にある

と定める。初期化・保持性・事後条件・停止性について確認してみよう。

1. 初期化：ループに入る前は、 $i = 0, j = n - 1$ であるため、正しい。
2. 保持性：ループ前に満たしていると仮定する。 $A[k] < x$ のとき、 $i = k$ に更新しているが、不変式は正しい。 $A[k] > x$ のとき、 $j = k - 1$ に更新しており、不変式は正しい。
3. 事後条件：ループが終了するのは $i > j$ のときである。このとき、不変式より、配列 A に x が存在するなら空の範囲 $A[i...j]$ の中にあることになる。したがって、 x は配列 A に存在せず、 -1 を返すことは正しい。
4. 停止性： $A[k] < x$ のとき $i = k$ に更新するため、探索範囲が変わらず無限ループに陥る可能性がある。 $i = j$ や $j = i + 1$ のとき、 $i = k$ のままである。

今回は、停止性に関して正しく動作しないことが確認できた。

3.1 二分探索クイズ

クイズ 3.1. 上の二分探索アルゴリズムが正しく・高速に動作するために必要な前提条件として正しいものをすべて選べ。

1. 配列が昇順にソートされていること。
2. 配列の要素がすべて異なること。
3. 配列の長さが2のべき乗であること。
4. 配列の各要素に計算量 $O(1)$ でアクセスできること。

クイズ 3.2. 二分探索と重複要素について正しいものをすべて選べ。

1. 配列に重複要素が含まれていても、昇順にソートされていれば二分探索は適用できる。
2. 配列に重複要素が含まれていると、二分探索は必ず失敗する。
3. 二分探索で見つかる要素は、重複する要素のうち最初の位置とは限らない。
4. 重複する要素の最初の位置を求めたい場合には、通常二分探索を少し変更する必要がある。

クイズ 3.3. 二分探索の正しさ、または時間計算量 $O(\log n)$ が保証されないケースとして正しいものをすべて選べ。

1. 配列がソートされていない.
2. 配列の各要素へのアクセスの計算量が $O(1)$ でない.
3. 比較関数の計算量が $O(1)$ でない.
4. 配列の長さが 2 のべき乗でない.

4 連結リスト・スタック・キュー

4.1 連結リスト

集合 $X = \{x_1, \dots, x_n\}$ の各要素を配列 A に格納すると、各要素を $A[i]$ のような形ですぐに呼び出せるが、一方で、 $A[i]$ と $A[i+1]$ の間に要素を追加したいときには、 $A[i+1]$ 以降の要素を一つずつずらしながら配列に格納し直す必要がある。このような手間の生じないデータ構造として**連結リスト (linked list)** がある。配列では添字 i の大きさに各要素の順序が決まるが、連結リストでは各ノードが指すポインタによって順序が決まる。

最も単純なのは、 i 番目のノードにおいて、要素 x_i と次のノードのポインタを格納する方法である。最初のノードは**先頭 (head) ノード**と呼ばれ (図 7 で 7 を格納するノード)、これを指すポインタは決まっている。末尾 (tail) にあるノードは NULL ポインタを指す。一つ前のノードのポインタも格納する場合は、**双方向連結リスト (doubly linked list)** と呼ばれる。

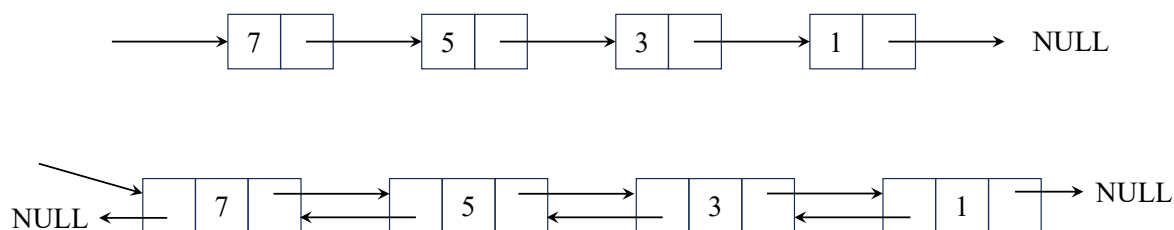


図 7 連結リスト

プログラムとしては、構造体を用いればノードを実現できる。

```

1 struct node {
2     int key;
3     struct node *next, *prev;
4 };

```

連結リストの応用例としてグラフの表現がある。 n 頂点のグラフは $n \times n$ 行列を用いて表現することができる。頂点間に辺があれば 1 なければ 0 とする行列でよく、**隣接行列 (adjacency matrix)** と呼ばれる。グラフが疎 (sparse)、つまり、辺の数が非常に少ないとき、隣接行列はほとんどの要素が 0 であり、数少ない 1 を表現するために、配列で n^2 個分の領域を確保しておくのはメモリ効率が悪い。このようなとき、連結リストを用いて、各頂点に対し、隣接する頂点を連結リストを用いて覚えておけば、辺の数程度の領域でグラフを表現できる。これは**隣接リスト (adjacency list)** 表現と呼ばれる。

連結リストでは、探索 (集合 X の中で鍵 k をもつ最初の要素を見つけ、そのポインタを返す)、挿入 (X に x を加える)、削除 (X から x を取り除く) などの操作ができることが期待される。探索は、辞書における Member の強化版である。これらの操作は以下の通り実現できる。

■**連結リスト上の探索** $\text{Search}(k)$ リストを順に辿り、鍵 k をもつ要素あればそのノードのポインタを返せばよい。計算量は $O(n)$ である。

1. 最初のノードを指すポインタを p とする。
2. $p = \text{NULL}$ ならば、 NULL を返して停止。
3. (そうでないとき) $p.\text{key} = k$ ならば、 p を返して停止。
4. (そうでないとき) $p = p.\text{next}$ とし、2に戻る。

■**連結リストにおける挿入** $\text{Insert}(x, b)$ (ノード b の次に x のノードを挿入) ノード b の次のノードとの間に新しいノードを加えればよい。 b のポインタが分かっているならば計算量は $O(1)$ であるが、 b を探す必要がある場合は一般に $O(n)$ にかかる。

1. 新しいノード a を用意し、その中に x を格納する。
2. b の次のノードを c とする。
3. $b.\text{next} = a$, $a.\text{next} = c$ となるようにポインタを更新する。

■**連結リストにおける削除** $\text{Delete}(x)$ (x のノードを削除) 要素 x を格納するノード a を見つける。 a が先頭ノードでなければ、その直前のノード b を見つけ、 b のポインタが a の直後のノードを指すようにすればよい。 a が先頭ノードであれば、先頭ノードを a の直後のノードに更新する。 a とその直前のノードのポインタが分かっているならば計算量は $O(1)$ であるが、一般にはそれらを探すために $O(n)$ にかかる。

4.2 スタックとキュー

連結リストにおける挿入と削除が先頭ノードでのみ行われる場合、**スタック** (*stack*) と呼ばれる。あとに挿入されたものほど先に削除されることから、後入れ先出し法 (LIFO (last-in first-out)) とも呼ばれる。関数の再帰呼び出し、数式の評価、履歴処理などで用いられる。スタックにおいて、挿入はプッシュ (push)、削除はポップ (pop) と呼ばれる。

連結リストにおいて、挿入が末尾だけで、削除が先頭だけで行われるとき、**キュー** (*queue*) と呼ばれる。先に挿入されたものから削除されるため、先入れ先出し法 (FIFO (first-in first-out)) とも呼ばれる。最初に来たものから処理をするときに用いられる。キューでは、挿入はエンキュー (enqueue)、削除はデキュー (dequeue) と呼ばれる。

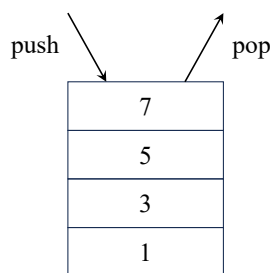


図8 スタック

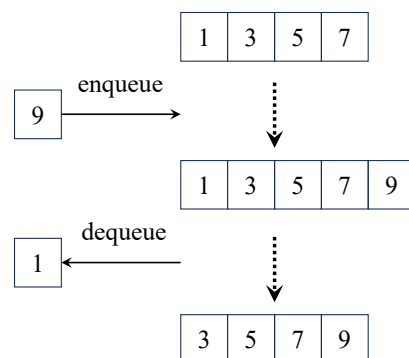


図9 キュー

スタックとキューは配列を用いて実現できる。スタックは、配列 S とポインタ変数 top を用いて、ス

タックが空なら $\text{top} = 0$ とし、要素 x を挿入するときは

```
top = top + 1; S[top] = x;
```

とし、削除は

```
top = top - 1;
```

とすればよい。

キューは、配列 Q とポインタ変数 front , back を用いて実現できる。ここでは、配列の添字を $0, 1, \dots, n-1$ とし、 $\text{front} = 0$, $\text{back} = 0$ と初期化する。要素 x の挿入は

```
Q[back] = x;  
back = (back + 1) % n;
```

とし、削除は

```
x = Q[front];  
front = (front + 1) % n;
```

とすればよい。これは環状の配列でキューを実現していることに相当する。ただし、このままではキューが空である場合と満杯である場合を区別しにくいので、保存されている要素数を別に管理するなどの工夫が必要である。

4.3 連結リスト・スタック・キュークイズ

クイズ 4.1. 配列と連結リストについて正しいものをすべて選べ。

1. 配列では、添字 i が分かっているならば $A[i]$ に $O(1)$ 時間でアクセスできる。
2. 連結リストでは、 i 番目の要素に常に $O(1)$ 時間でアクセスできる。
3. 連結リストでは、挿入したい位置の直前のノードが分かっているならば、その直後への挿入は $O(1)$ 時間でできる。
4. 配列では、途中に要素を挿入するとき、後ろの要素をずらす必要が生じることがある。

クイズ 4.2. スタックとキューについて正しいものをすべて選べ。

1. スタックでは、最後に挿入された要素が先に取り出される。
2. キューでは、最初に挿入された要素が先に取り出される。
3. スタックは FIFO、キューは LIFO と呼ばれる。
4. 配列を用いてスタックやキューを実現できる。

5 二分探索木

辞書における演算 Member, Insert, Delete は連結リストで実現できるが、どの操作も計算量が $O(n)$ であった。一方、二分探索を用いれば、ソート済み配列に対して Member を $O(\log_2 n)$ で実現できる。しか

し、配列の途中に要素を挿入したり、途中の要素を削除したりするには、要素をずらす必要があり、一般には $O(n)$ 時間かかる。二分探索の考え方を木構造に取り入れ、探索・挿入・削除を統一的に扱うデータ構造が**二分探索木** (binary search tree) である。

二分木として、どのノードも子の数が 2 または 0 であり、左と右の子が区別されている根付き木を考える。このような木は**全二分木** (full binary tree) と呼ばれ、完全二分木 (complete binary tree) と区別される。全二分木において、子を 2 つもつ (葉でない) ノードを**内点** (internal node) と呼び、子をもたない (葉である) ノードを**外点** (external node) と呼ぶ。全二分木では、内点の数が n であれば、外点の数は $n + 1$ である。

5.1 対称順と二分探索木上の操作

二分探索木では、集合 X の要素は全二分木の内点のみに割り当てられ、外点は NULL が割り当てられる。ここでは、 X の要素の鍵を二分木の各内点に割り当てることを考え、 X の各要素の鍵の値はすべて異なることを仮定する。内点 v に割り当てた鍵を $v.key$ と表したとき、二分探索木では以下の対称順 (symmetric order) を満たすように割り当てる。

対称順： 各内点 v に対し、左の子 $v.lc$ が内点ならば $v.lc$ の子孫で内点であるすべての u に対し、 $u.key < v.key$ であり、右の子 $v.rc$ が内点ならば $v.rc$ の子孫で内点であるすべての w に対し $v.key < w.key$ である。

つまり、対称順であれば、各ノードの左側に鍵の小さなノード、右側に鍵の大きなノードが配置される。二分探索木は、全二分木のうち、内点に割り当てられた鍵が対称順を満たすものと定義する (図 10 参照)。

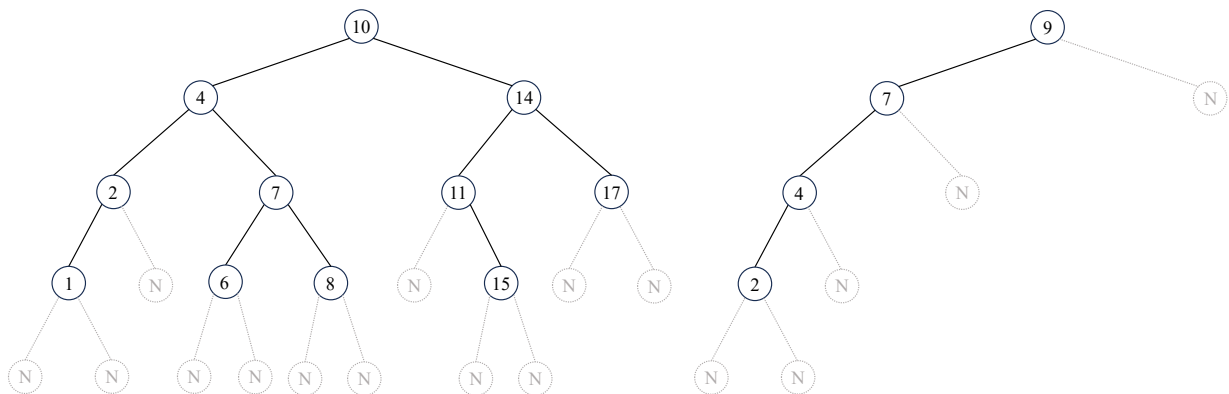


図 10 二分探索木

Search, Insert, Delete の実行は以下の通りである。

■二分探索木における探索 Search(k) (鍵 k を割り当てたノード v を返す)

1. 木の根を v とする。
2. ノード v が外点ならば $v (= \text{NULL})$ を返して停止。
3. (そうでないとき) $v.key = k$ ならば v を返して停止。
4. (そうでないとき) $v.key < k$ ならば $v = v.rc$ とし、そうでなければ $v = v.lc$ とし、2 に戻る。

■二分探索木における挿入 Insert(k) (鍵 k のノードを挿入する)

1. 木の根を v とする。

2. ノード v が外点ならば, v を内点にし, $v.key = k$ とする. さらに, v の左の子と右の子として外点を追加して停止.
3. (そうでないとき) $v.key < k$ ならば $v = v.rc$ とし, そうでなければ $v = v.lc$ とし, 2 に戻る.

■二分探索木における削除 Delete(k) (鍵 k を割り当てたノードを削除する)

1. Search(k) を実行し, $u.key = k$ である内点 u を見つける.
2. u の子の少なくとも片方が外点のとき, もう片方を c とする. u の親 $p(u)$ が存在すれば, u の代わりに c を $p(u)$ の子とする. u が根であれば, c を新しい根とする. (これで内点は削除された)
3. u の子がいずれも内点のとき, 以下を行う.
 - (a) $d = u.lc$ とし, $d.rc$ が内点である限り $d = d.rc$ を繰り返し行う. (結果, d は鍵が k 未満で最大の内点)
 - (b) $u.key = d.key$ とする.
 - (c) $u = d$ とし, 2 に戻る. (これにより, 内点 d が削除される)

Min, Max と Predecessor, Successor は以下の通り実現できる.

■二分探索木における Min (鍵最小のノード v を返す)

1. 木の根を v とする.
2. v の左の子が内点である限り $v = v.lc$ とする.
3. v を返して停止.

Max (鍵最大のノードを返す) も同様に実現できる.

■二分探索木における Predecessor(v) (鍵の小さい順でノード v の直前ノード u を返す)

1. v の左部分木が空でないとき, 左部分木に対して Max を適用した結果を返す.
2. そうでないとき, 根に向かって親ノードをたどり, 右の子の子孫に v を含むような最初のノードを返す. (その頂点 u に対し, v は右の子の子孫であるため $u.key < v.key$ であり, v から最初にたどったノードはそのうち最大の鍵をもつ.)

ただし, そのようなノードが存在しない場合, Predecessor(v) は存在しない.

Successor(v) (鍵の小さい順でノード v の直後ノード u を返す) も同様に実現できる.

各操作の計算量は, いずれも木の高さに比例する. したがって, 一般的には $O(n)$ しか保証できない.

5.2 二分探索木の平均的な振る舞い

集合の各要素の鍵が $x_1, x_2, \dots, x_n$ の順に挿入され, $x_1 < x_2 < \dots < x_n$ を満たしていたとき, できた二分木は高さ $n - 1$ のリストになり, 挿入や削除は計算量 $O(n)$ にかかる. ただ, このような状況は稀にしか起きないように見える. そこで, $x_1, x_2, \dots, x_n$ がランダムな順番に挿入された場合の, 探索の計算量を分析してみよう.

ランダムな順に n 個を挿入してできた二分探索木における内点のパス長の総数の期待値を $S(n)$ とする. 期待値は, $n!$ 通りの挿入が等確率で起きたときを考える. まず, $S(0) = S(1) = 0$ である. 内点数 n の任意の二分探索木において, ある $0 \leq i \leq n - 1$ が存在し, 根の左部分木に i 個, 右部分木に $n - i - 1$ 個内点がある. 仮定より, i は等確率に選ばれる. 固定した i に対し, $S(i)$ は左部分木の内点のパス長の総数の期待値であり, $S(n - i - 1)$ は右部分木の方に対応する. 根があることで, その他 $n - 1$ 個の内点に長さ 1

ずつパス長が足される。したがって、 $S(n) = n - 1 + S(i) + S(n - i - 1)$ という漸化式が成り立つ。この式を $0 \leq i \leq n - 1$ すべてについて足し合わせると、

$$S(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} S(i)$$

が得られる。 $S(n - 1)$ の場合との差をとれば、

$$\frac{S(n)}{n+1} = \frac{S(n-1)}{n} + \frac{4}{n+1} - \frac{2}{n}$$

と計算できる。 $S(n - 1)/n$ の項を展開すれば、

$$\begin{aligned} \frac{S(n)}{n+1} &= \frac{S(0)}{1} + 4 \left(\frac{1}{n+1} + \frac{1}{n} + \cdots + \frac{1}{2} \right) - 2 \left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{1} \right) \\ &= S(0) + 4(H_{n+1} - 1) - 2H_n \end{aligned}$$

である。ここで、 $H_n = \sum_{i=1}^n 1/i$ は調和数 (harmonic number) である。不等式 $\ln(n+1) \leq H_n \leq 1 + \ln n$ が成り立つため、 $S(n) = O(n \log_2 n)$ が導かれる。

したがって、ランダムな順に挿入して二分探索木をつくれば、平均的な探索計算量は $O(\log_2 n)$ であり、平衡した二分探索木での探索計算量と同程度である。

5.3 平衡探索木

二分探索木の操作 Search, Insert, Delete の計算量はいずれも木の高さに依存するため、二分木の高さを $O(\log_2 n)$ に保つことができれば効率的に実行できる。それを実現する**平衡探索木** (*balanced search tree*) としては、AVL 木、2-3 木、B 木、赤黒木などが知られている。木の更新操作 (Insert, Delete) の後に平衡でなくなる場合があり、その際に再平衡化を行う必要がある。ここでは、AVL 木と赤黒木 (二色木とも呼ばれる) を簡単に紹介する。

5.3.1 AVL 木

AVL 木は以下の平衡条件を満たす二分探索木である。

任意のノードの左の部分木と右部分木の高さの差は高々 1 である。

この条件により、AVL 木の高さは $\Theta(\log_2 n)$ を保つことができる。より強い平衡条件をもつ完全二分木は、その制約のため、変更操作のあとに条件を保つことが難しい。制約を少し緩和した AVL 木では、再平衡化が可能である。しかし、再平衡化のためにある程度の追加計算が必要である。

再平衡化の計算量が比較的小さいものとして、赤黒木が知られている。

5.3.2 赤黒木

赤黒木は以下の平衡条件を満たす全二分木である。

1. 各ノードは赤色か黒色のいずれかである。
2. 赤ノードの親は必ず黒ノードであり、根と外点は黒である。
3. 根から外点のパスはいずれも、同数の黒ノードを含む。

この平衡条件より、根から外点へのパスで最短のものと最長のものは長さが高々 2 倍以下である。これより、木の高さは $O(\log_2 n)$ である。より具体的に、外点の深さの最小値を d としたとき、木の高さ h

は $h \leq 2d$ を満たす. 定義より, 深さ $c < d$ のノードは内点で子を 2 つもつため, 深さ c の内点の個数は 2^c であり, 赤黒木における内点の総数は, $2^0 + 2^1 + \dots + 2^{d-1} = 2^d - 1$ 以上である. これより, $h \leq 2d \leq 2\lceil \log_2(n+1) \rceil = O(\log_2 n)$ である.

5.4 探索木クイズ

クイズ 5.1. 以下のうち正しいものをすべて選べ.

1. n 個の内点をもつ二分探索木の高さは必ず $\Omega(\log_2 n)$ である.
2. ヒープは二分探索木の特殊な場合である.
3. 二分探索木では, どのノードについても, 左部分木の鍵はそのノードの鍵より小さく, 右部分木の鍵はそのノードの鍵より大きい.
4. 二分探索木の高さは常に $O(\log_2 n)$ である.

クイズ 5.2. 二分探索木において, 次の操作のうち, 木の高さが $O(\log_2 n)$ ならば $O(\log_2 n)$ で実現できるものをすべて選べ.

1. 最小値の探索
2. 最大値の探索
3. 任意の値の探索
4. k 番目に小さい値の探索

クイズ 5.3. 二分探索木において, 鍵 363 を探索したときに調べるノードの列として起こり得ないものをすべて選べ.

1. 2, 252, 401, 398, 330, 344, 397, 363.
2. 924, 220, 911, 244, 898, 258, 362, 363.
3. 925, 202, 911, 240, 912, 245, 363.
4. 2, 399, 387, 219, 266, 382, 381, 278, 363.
5. 935, 278, 347, 621, 299, 392, 358, 363.

クイズ 5.4. 平衡二分探索木 (例: AVL 木, 赤黒木) の利点として正しいものをすべて選べ.

1. 探索・挿入・削除の最悪計算量を $O(\log_2 n)$ にできる.
2. 常に完全二分木になる.
3. 挿入・削除後も高さが $O(\log_2 n)$ に保たれる.
4. メモリ使用量が $O(1)$ である.

6 ハッシュ表

二分探索木は Search, Insert, Delete が木の高さに比例する計算量ででき, 平衡探索木である赤黒木は木の高さを $O(\log_2 n)$ に保つ方法であった. ここでは, これらの操作を平均的に $O(1)$ の計算量で実現する **ハッシュ表** (hash table) を見てみよう.

集合 $X = \{x_1, \dots, x_n\}$ の各要素 x_i の鍵 k_i は集合 U の要素であり, $n \ll |U|$ である場合を想定する. つまり, 各鍵は広い空間から選ばれているが, U の要素数程度の配列を確保してしまうと, メモリ効率が

悪くなってしまう。

ハッシュ表は、サイズ m の配列 T とハッシュ関数 $h : U \rightarrow [m]$ を用いる方法である ($[m] := \{0, 1, \dots, m-1\}$)。鍵 $k_i \in U$ の代わりに $h(k_i)$ を用いることにし、要素 x_i は配列 $T[h(k_i)]$ に格納する。問題が生じるのは、 $k_i, k_j \in U$ が $h(k_i) = h(k_j)$ を満たすときであり、これは**衝突 (collision)** と呼ばれる。衝突への対処法として、チェーン法 (chaining) とオープンアドレス法 (open addressing) がよく知られている。

6.1 チェーン法

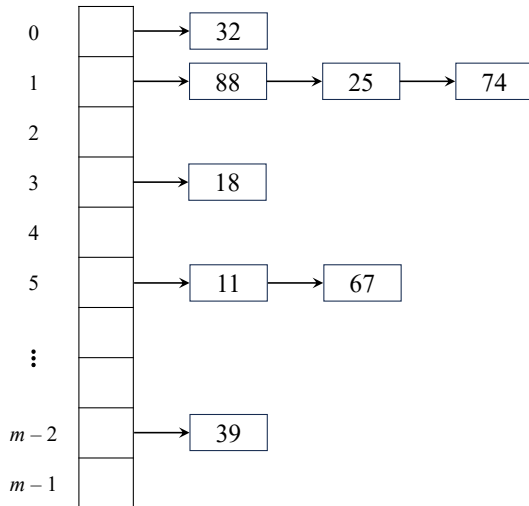


図 11 チェーン法

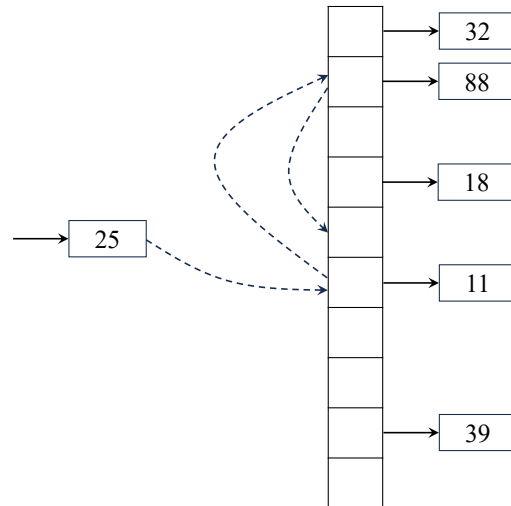


図 12 オープンアドレス法

チェーン法は、衝突が起きた場合に、衝突したすべての要素を連結リストとして格納する方法である。

探索 $\text{Search}(k)$ は、 $h(k)$ を求め、 $T[h(k)]$ が指す連結リストにおいて k を探索すればよい。Insert や Delete も連結リストを用いて同様に実現できる。いずれの操作も計算量は連結リストの長さに比例する。ハッシュ関数に対し、単統一様性という性質を仮定すれば、各操作の計算量は $O(1)$ であることがわかる。

単統一様性 : 任意の $i \in [n], a \in [m]$ に対し、 $\Pr(h(k_i) = a) = \frac{1}{m}$

上記仮定のもと、 i 回操作を行ったあと、どの連結リストも長さの期待値は i/m 以下であり、 $(i+1)$ 回目の操作の計算量の期待値は $O(i/m + 1)$ である。 ℓ 回操作したときの計算量の期待値の和は

$$\sum_{i=1}^{\ell} O\left(\frac{i}{m} + 1\right) = O\left(\frac{\ell(\ell-1)}{2m} + \ell\right)$$

であり、1 回あたりの期待値は $O((\ell-1)/2m + 1)$ である。したがって、 n 個の要素を格納したあとの操作の計算量の期待値は $O(n/m + 1)$ である。これは、 $n = O(m)$ であれば計算量 $O(1)$ であることを意味する。

6.2 オープンアドレス法

オープンアドレス法は、衝突が起きたとき、次の場所を探しに行く方法である。次に探索する場所を決める方法はいくつかあり、線形探査法 (linear probing) では、 $T[h(k)]$ の次に $T[(h(k) + 1) \bmod m]$ 、その次に $T[(h(k) + 2) \bmod m]$ というように順に探していく。二重ハッシュ (double hashing) では、探索

のオフセットを計算するためのハッシュ関数を用意する。例えば、 $h_1(k) = 40, h_2(k) = 13$ のとき、まず $T[40]$ を探し、衝突が起きるときは $T[53]$ を探す。そこでも衝突する場合は $T[66]$ を探す、というのを繰り返す。

6.3 ハッシュ表の応用

■**重複排除 (de-duplication)** 大量のデータを順番に処理するとき、重複するデータを排除したい場合がある。このようなとき、ハッシュ表を使ってすでに存在するかどうかを確認すれば、重複を見つけることができる。

■**2-SUM 問題** 与えられた整数の集合 $S = \{s_1, s_2, \dots, s_n\}$ に対し、 $s_i + s_j = t$ を満たす i, j を見つける問題である。すべての組み合わせを調べれば計算量 $O(n^2)$ で解ける。もう少し賢い方法として、 S の要素を計算量 $O(n \log_2 n)$ でソートし、 $i = 1, \dots, n$ に対し $t - s_i$ が S に存在するかを二分探索で調べれば計算量 $O(n \log_2 n)$ で解くことができる。ハッシュ表を使えば計算量 $O(n)$ で解くことができる。 $i = 1, \dots, n$ の順に、まず $t - s_i$ がすでにハッシュ表に存在するかを調べ、存在すればその組を出力する。その後、 s_i をハッシュ表に格納する。

6.4 ユニバーサルハッシュ関数

ハッシュ関数 $h : U \rightarrow [m]$ を関数集合からランダムに選ぶことを許せば、以下の性質が実現できる。

定義 6.1. 関数集合 $\mathcal{H} = \{h : U \rightarrow [m]\}$ が**ユニバーサル (universal)** であるとは、任意の異なる $u, v \in U$ に対し、

$$\Pr_{h \in \mathcal{H}}(h(u) = h(v)) \leq \frac{1}{m}$$

を満たすことである。ただし、関数 h は $\mathcal{H}$ から一様ランダムに選ばれる。

関数 $h : U \rightarrow [m]$ をユニバーサルハッシュ関数 $\mathcal{H}$ からランダムに選び、サイズ m の配列 T を使ったチェーン法により n 個の（異なる）鍵を格納したとする。このとき、ハッシュ表の連結リストの長さの期待値は $n/m + 1$ 以下であることがわかる。以下がその証明である。

異なる鍵 $k_i, k_j \in U$ に対し、ユニバーサルハッシュ関数の性質より、 $\Pr(h(k_i) = h(k_j)) \leq 1/m$ である。 n 個の鍵 $k_1, \dots, k_n$ を格納したあと、ある鍵 k と同じ隣接リストに入る鍵の数を X_k とする。鍵 k_i が k と同じ隣接リストに入ることを表す確率変数を Y_i とすると ($Y_i = \mathbb{I}(h(k_i) = h(k))$)、 $X_k = \sum_{i=1}^n Y_i$ であり、 $\mathbb{E}[Y_i] \leq 1/m$ である。したがって、

$$\mathbb{E}[X_k] = \mathbb{E}\left[\sum_{i=1}^n Y_i\right] = \sum_{i=1}^n \mathbb{E}[Y_i] \leq \frac{n}{m}$$

が導かれる。上記は、鍵 k が $k_1, \dots, k_n$ に含まれない場合に相当する。鍵 k が含まれている場合、 Y_i の一つが 1 であり、 k を含む連結リストの長さの期待値は $n/m + 1$ である。

ユニバーサルハッシュ関数の実現方法は様々なものが知られている。

■**数論にもとづく構成法** まず、 $|U| < p$ を満たす素数 p を選ぶ。 $\mathbb{Z}_p := \{0, 1, \dots, p-1\}$, $\mathbb{Z}_p^* := \{1, 2, \dots, p-1\}$ とし、ここでは $U \subseteq \mathbb{Z}_p$ と考える。任意の $a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p$ に対し、関数 $h_{a,b} : \mathbb{Z}_p \rightarrow [m]$ を、

$$h_{a,b}(u) = ((au + b) \bmod p) \bmod m$$

と定める。

定理 6.2. 関数集合 $\mathcal{H} = \{h_{a,b} \mid a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}$ はユニバーサルである。

証明. 異なる $u, v \in \mathbb{Z}_p$ が衝突するのは,

$$(au + b) \bmod p \equiv (av + b) \bmod p \pmod{m}$$

を満たすときである. ここで, $x = (au + b) \bmod p, y = (av + b) \bmod p$ とおくと, 必ず $x \neq y$ であることがわかる. さらに, $a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p$ を一様ランダムに選ぶとき, (x, y) は $\mathbb{Z}_p \times \mathbb{Z}_p$ 上の異なる 2 点を等確率でとる. したがって, 異なる u, v が衝突する確率は, $\mathbb{Z}_p \times \mathbb{Z}_p$ 上の異なる 2 点が法 m のもとで等しい確率である. $x \in \mathbb{Z}_p$ が与えられたとき, $x \equiv y \pmod{m}$ かつ $y \neq x$ を満たす $y \in \mathbb{Z}_p$ の数は, 高々

$$\left\lceil \frac{p}{m} \right\rceil - 1 \leq \frac{p-1}{m}$$

個である. したがって, 異なる順序対 (x, y) で $x \equiv y \pmod{m}$ を満たすものは高々 $p(p-1)/m$ 個である. 以上より, 異なる $u, v \in \mathbb{Z}_p$ が衝突する確率は

$$\frac{p(p-1)/m}{p(p-1)} = \frac{1}{m}$$

以下であり, $\mathcal{H}$ がユニバーサルであることが示された. □

6.5 Bloom フィルタ

Bloom フィルタは, Search と Insert をメモリ効率よく実現する方法である. ハッシュ表と比較すると, どちらの操作も計算量 $O(1)$ を保証するが, 集合 S に属さない x を $x \in S$ と誤判定 (false positive) することがあり, また, オブジェクトへのポインターを格納することができない.

Bloom フィルタは, m ビットの配列 B と, k 個のハッシュ関数 $h_1, \dots, h_k : U \rightarrow [m]$ を用いる. 要素 x を追加するには, $i = 1, \dots, k$ に対し $B[h_i(x)]$ を 1 にする. 要素 x が S に属するかどうかを調べるには, すべての $i = 1, \dots, k$ に対し $B[h_i(x)] = 1$ であるか調べる. すべて 1 であれば, $x \in S$ と判定し, そうでなければ $x \notin S$ と判定する. Bloom フィルタの計算量は, k 個のハッシュ関数を評価するため, $k = O(1)$ であれば計算量 $O(1)$ である.

■Bloom フィルタの応用例 禁止パスワードリストとの照合に使うことができる. Bloom フィルタでは false positive は許すが, false negative は許さないため, このような使い方ができる. 同様の理由で, スペルチェックにおいて, 存在しない単語をチェックするために使うことができる. インターネットルータで過去に通過したパケットかどうかを高速にチェックするためにも使われる.

6.6 ハッシュクイズ

クイズ 6.1. 誕生日がランダムに与えられた n 人がいたとき, 同じ誕生日の人がいる確率が 80% を超えるために必要な n の大きさは次のうちどれ (誕生日は 365 日から選ぶものとする).

1. 23

2. 35

3. 47

4. 57

クイズ 6.2. ハッシュ関数として, 関数集合 $\mathcal{H} = \{h : U \rightarrow [m]\}$ からランダムに選んだ関数を使うことはなぜ実用的でないのか. 理由として適切なものをすべて選べ.

1. 実際のところは実用的である.
2. 決定性の関数でないから.
3. 関数を保存するための容量が大きすぎる.
4. 関数を評価するための時間が大きすぎる.

クイズ 6.3. 以下の各ハッシュ関数がユニバーサルかどうか答えよ.

1. 集合 $\mathcal{H} = \{h_{a_1, a_2} \mid a_1, a_2 \in [m]\}$. ただし, m は素数であり,

$$h_{a_1, a_2}(x_1, x_2) = a_1 x_1 + a_2 x_2 \bmod m$$

で定義される関数 $h_{a_1, a_2} : [m]^2 \rightarrow [m]$ の集合.

2. 上記と同様の $\mathcal{H}$ に対し, 整数 k を用いて $m = 2^k$ としたもの.
3. 関数 $f : [m] \rightarrow [m-1]$ 全体からなる集合.

クイズ 6.4. Bloom フィルタについて正しいものをすべて選べ.

1. 要素 x が実際には集合 S に含まれないのに, $x \in S$ と判定されることがある.
2. 要素 x が実際には集合 S に含まれるのに, $x \notin S$ と判定されることがある.
3. 要素そのものや, その要素へのポインタを取り出す用途には向いていない.
4. $k = O(1)$ 個のハッシュ関数を用いれば, 探索と挿入は $O(1)$ 時間で実行できる.

7 データ構造の演習問題

演習問題 7.1. n 個の内点からなる二分探索木は $n+1$ 個の外点をもつことを示せ.

演習問題 7.2. n 個の内点をもつ全二分木が完全二分木となるための必要十分条件を示し, そのときの木の高さ h と n の関係式を導け. また, n が任意のとき, 全二分木が完全二分木でない場合, その最小の高さと最大の高さを求めよ.

演習問題 7.3. 二分探索木における鍵の探索順として起き得ないものを以下よりすべて選べ.

1. 2, 252, 401, 398, 330, 344, 397, 363.
2. 924, 220, 911, 244, 898, 258, 362, 363.
3. 925, 202, 911, 240, 912, 245, 363.
4. 2, 399, 387, 219, 266, 382, 381, 278, 363.
5. 935, 278, 347, 621, 299, 392, 358, 363.

演習問題 7.4. 二分探索木のあるノード v の子の数が 2 のとき, 鍵の小さい順で v の直前ノード (predecessor) は右の子をもたず, v の直後ノード (successor) は左の子をもたないことを示せ.

演習問題 7.5. 二分探索木 T の各ノードの鍵の値が異なり, x は葉ノード, y はその親ノードとする. このとき, $y.key$ は, T の中で $x.key$ より大きな値の中で最小の鍵, あるいは, T の中で $x.key$ より小さな値の中で最大の鍵のいずれかである. これを示せ.

演習問題 7.6. n 個の異なる整数からなる配列 A が与えられているとき, A を二分探索木に挿入する順序として, 最も高さが小さくなる順序と, 最も高さが大きくなる順序をそれぞれ説明せよ. また, そのときの木の高さを n で表せ.

演習問題 7.7. ヒープに対して, すべての要素の値を k だけ増やす (または減らす) 操作を $O(1)$ で実現す

る方法を考えよ。また、この方法を用いた場合、FindMin や Insert などの操作にどのような影響があるか説明せよ。

演習問題 7.8. 連結リストで実現された集合 X に対し、要素 x の削除を $O(1)$ で実現するためにはどのような追加情報が必要か説明せよ。また、その場合の挿入・削除・探索の計算量を答えよ。

演習問題 7.9. 二分探索木において、各ノードに部分木のサイズ（そのノードを根とする部分木に含まれるノード数）を持たせることで、 k 番目に小さい要素の探索を $O(\log n)$ で実現するアルゴリズムを設計せよ。

演習問題 7.10. ノード数 n の二分木の総数を b_n とする。

1. $b_0 = 1$ のとき、 $n \geq 1$ について

$$b_n = \sum_{k=0}^{n-1} b_k b_{n-1-k}$$

が成り立つことを示せ。

2. 母関数 $B(x)$ を、

$$B(x) = \sum_{n=0}^{\infty} b_n x^n$$

と定める。 $B(x) = xB(x)^2 + 1$ であることを示せ。これより、 $B(x)$ を閉じた形で表す一つの方法は、

$$B(x) = \frac{1 - \sqrt{1 - 4x}}{2x}$$

である。 $\sqrt{1 - 4x}$ をマクローリン展開すると $B(x)$ が求まり、

$$b_n = \frac{1}{n+1} \binom{2n}{n}$$

が導かれる。この数は**カタラン数** (Catalan number) と呼ばれ、二分木の総数以外にも、様々な組み合わせ的解釈をもつ数である。

クイズの答え

クイズ 2.1 1, 2, 3 が正しい。最小ヒープでは根が最小であり、親の鍵は子の鍵以下である。また、最大値をもつ要素は葉のどこかにある。一方、配列表現がソート済みになるとは限らない。

クイズ 2.2 2 が正しい。FindMin は $O(1)$ 、Insert・DeleteMin・DecreaseKey は $O(\log_2 n)$ 、ヒープ構成は $O(n)$ である。

クイズ 2.3 1 が正しい。10 番目に小さい要素は、DeleteMin を定数回繰り返せば得られる。最大値や中央値を見つけるには、一般にはヒープ中の多くの要素を調べる必要がある。

クイズ 3.1 正解は 1, 4 である。二分探索では、中央の要素と比較して探索範囲の半分を捨てるため、配列が昇順にソートされている必要がある。また、各ステップで中央の要素に直接アクセスするので、配列の各要素に $O(1)$ でアクセスすることも必要である。重複要素があってもよく、配列の長さが 2 のべき乗である必要もない。

クイズ 3.2 正解は 1, 3, 4 である。重複要素があっても、配列がソートされていれば二分探索は適用できる。ただし、通常の二分探索では、重複する要素のうちどの位置が見つかるかは実装に依存する。最初の位置や最後の位置を求めたい場合には、見つかったあとも探索範囲をさらに絞るように変更する必要がある。

クイズ 3.3 正解は 1, 2, 3 である。配列がソートされていなければ、探索範囲の半分を捨てる判断が正しいとは限らない。また、配列要素へのアクセスや比較に $O(1)$ より大きな時間がかかる場合、全体の計算量もその分だけ大きくなる。一方、配列の長さが 2 のべき乗でなくても、探索範囲は毎回ほぼ半分になるため問題ない。

クイズ 4.1 正解は 1, 3, 4 である。配列は添字による直接アクセスができるため、 $A[i]$ を $O(1)$ 時間で読める。一方、連結リストでは先頭からポインタをたどる必要があるため、 i 番目の要素へのアクセスは一般には $O(i)$ 時間かかる。ただし、挿入位置の直前のノードが分かっているならば、ポインタのつなぎ替えだけで挿入できる。

クイズ 4.2 正解は 1, 2, 4 である。スタックは後入れ先出し法 LIFO、キューは先入れ先出し法 FIFO に従う。したがって、3 は逆である。どちらも連結リストでも配列でも実現できる。

クイズ 5.1 正解は 1, 3 である。 n 個の内点をもつ二分木の高さは、どれだけ平衡していても $\Omega(\log_2 n)$ である。また、二分探索木では、左部分木の鍵は小さく、右部分木の鍵は大きい。一方、ヒープは親子間の大小関係だけを満たす構造であり、二分探索木とは別物である。また、二分探索木は挿入順によってリスト状になることがあり、高さは常に $O(\log_2 n)$ とは限らない。

クイズ 5.2 正解は 1, 2, 3 である。最小値・最大値・任意の値の探索は、根から葉への 1 本のパスをたどればよいので、木の高さに比例する時間で実行できる。一方、通常の二分探索木では、各部分木のサイズを保存していないため、 k 番目に小さい値をただちに判断できない。そのため、4 は一般には $O(\log_2 n)$ では実現できない。

クイズ 5.3 正解は 3, 5 である。二分探索木の探索では、探索中に「候補となる鍵の範囲」が更新される。3 では、911 から左に進んだ後なので、それ以降の鍵は 911 より小さくなければならない。しかし、その後に 912 が現れるため、この探索順は起こり得ない。5 では、347 から右に進んだ後なので、それ以降の鍵は 347 より大きくななければならない。しかし、その後に 299 が現れるため、この探索順は起こり得ない。

クイズ 5.4 正解は 1, 3 である。平衡二分探索木は、挿入・削除の後に必要に応じて木を再構成し、高さを $O(\log_2 n)$ に保つ。そのため、探索・挿入・削除の最悪計算量を $O(\log_2 n)$ にできる。ただし、常に完全二分木になるわけではなく、ノード数に比例する記憶領域は必要である。

クイズ 6.1 正解は 2 である。誕生日が全員異なる確率は

$$\frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - n + 1}{365}$$

であり、その余事象が同じ誕生日の人がいる確率である。 $n = 35$ のとき、この確率は 80% を超える。

クイズ 6.2 正解は 3, 4 である。ランダムな関数そのものを保存するには、 U の各要素に対する値を覚える必要があり、容量が大きすぎる。また、関数を明示的に保存しない場合、評価を効率よく行う方法も明らかでない。実用上は、少ないパラメータで表せるハッシュ関数族を使う。

クイズ 6.3 正解は 1 のみである。1 は m が素数なので、異なる入力に逆元を使うことができ、衝突確率を $1/m$ 以下に抑えられる。2 は $m = 2^k$ であり、差が偶数成分をもつ場合などに衝突確率が大きくなることもある。3 は出力先が $m - 1$ 個しかないため、衝突確率は $1/(m - 1)$ となり、 $1/m$ 以下にはならない。

クイズ 6.4 正解は 1, 3, 4 である。Bloom フィルタでは、含まれていない要素が含まれていると誤判定する false positive は起こり得る。一方、挿入済みの要素については、対応するビットがすべて 1 になっているため false negative は起こらない。また、Bloom フィルタは所属判定用の構造であり、要素そのものを取り出す用途には向いていない。